

操作系统

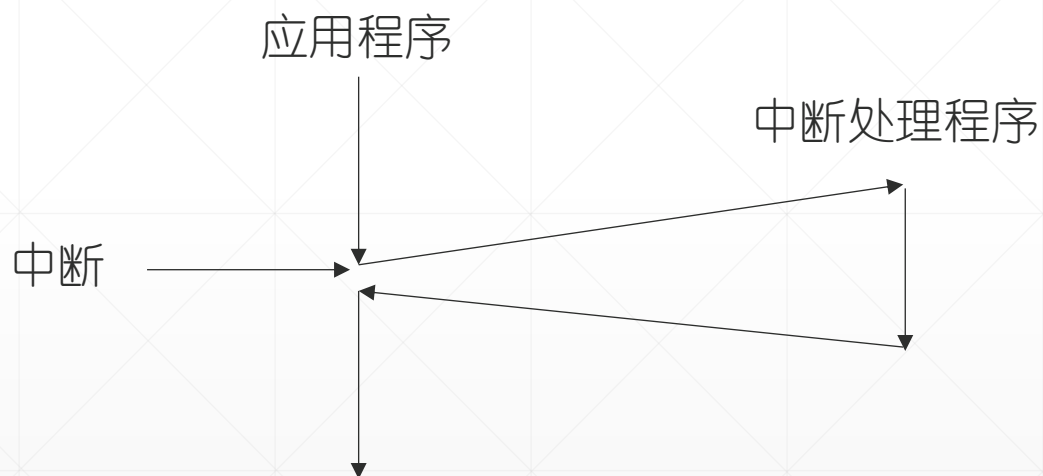
第二章 进程管理

第二部分 (2) Unix V6++的中断系统

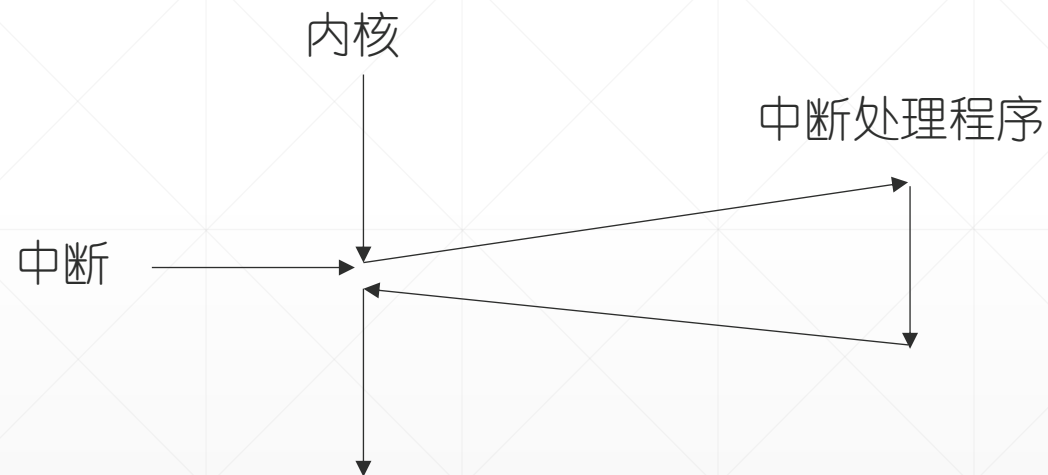
同济大学计算机系

一、中断

- 中断，是CPU必需响应的信号。
- CPU响应中断，暂停当前任务，执行中断处理程序。
- 中断处理程序运行结束后，CPU恢复执行被中断任务。



图：现运行进程执行应用程序时被中断。暂停执行应用程序，陷入内核（用户态切换至核心态），执行中断处理程序。中断处理程序执行完毕后，返回用户态，继续执行应用程序。



图：现运行进程执行内核代码时被中断。暂停执行当前任务，执行中断处理程序。中断处理程序执行完毕后，继续执行被打断的任务。

二、中断请求（中断源 和 中断向量号）

CPU支持的中断有很多种，

- 中断源是发出中断信号的实体。
- 中断号（中断向量号），是标识中断源的唯一整数。

根据中断源，中断分3类

- 内部中断，是CPU发给自己的中断信号。
- 外部中断，是 时钟芯片 或 外设芯片发来的中断信号。
- 处理器间中断，是其它CPU发来的中断信号。

内部中断

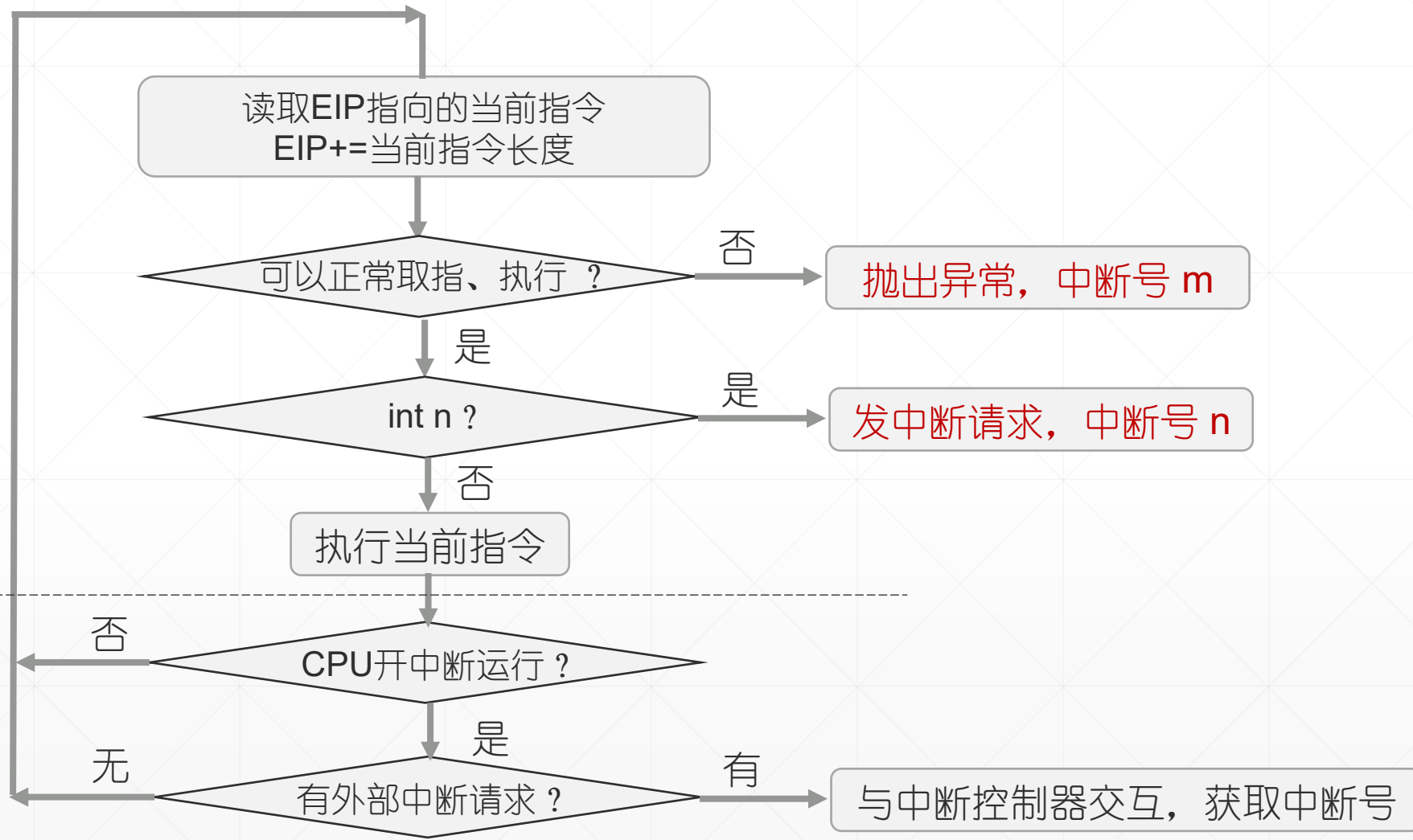
- 内部中断是CPU发给自己的中断请求信号。通知执行单元暂停执行下条指令。
- 硬件发出的中断请求，叫**异常**。CPU抛出异常是因为它无法顺利执行当前任务（取下条指令 或 执行当前指令），有3种可能：
 - (1) 硬件故障
 - (2) 程序bug，比如 非法指令、运算错误（除数是0）、非法内存访问
 - (3) 现运行进程需要追加存储单元
 - 扩展堆栈
 - 虚拟存储器，请求调页
- 应用程序发出的中断请求，有2种可能
 - (1) 请求执行**系统调用** int指令 或 syscall指令
 - (2) 被调试的程序走到断点，请求暂停执行



外部中断

- 外部中断，是外设芯片 或 时钟芯片发给CPU芯片的中断请求信号。
 - **外设中断**是外设芯片发出的中断信号，表示IO完成。
 - **时钟中断**是时钟芯片发出的中断信号。内核使用这个均匀的脉冲信号
 - (1) 维护系统时钟
 - (2) 实现基于时间的调度策略，比如时间片轮转
 - (3) 提供定时器服务

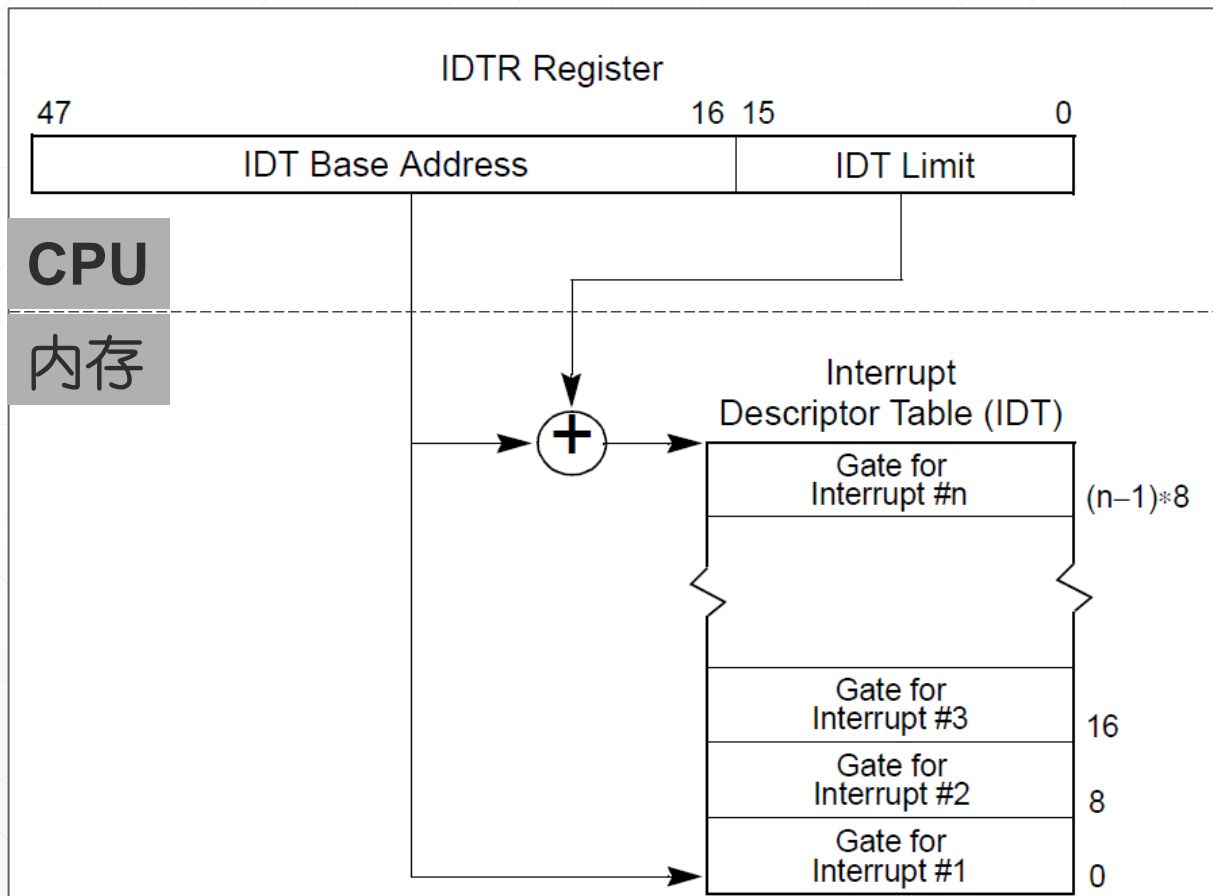
继续执行当前子程序



三、中断向量表

- 每种中断，对应一个中断处理程序。
- 内存中，有一个中断向量表。有 n 个元素，每个元素是一个中断描述符，登记中断处理程序的入口地址。
- 响应中断请求时，CPU硬件用中断号查中断向量表，获得入口地址，赋值PC (Program Counter, 存放下条指令的地址)。跳转、执行中断处理程序。

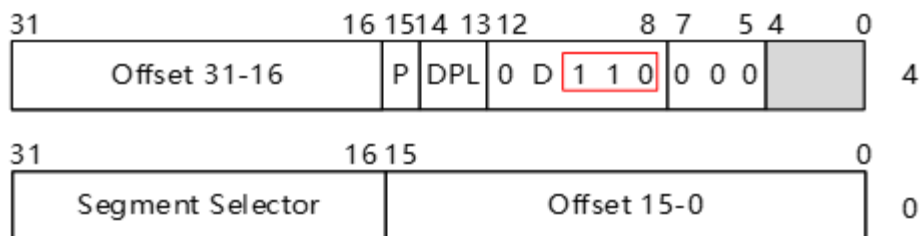
Unix V6++的中断向量表, IDT(Interrupt Descriptor Table)



- 内存中, 有一个中断描述符表, 简称IDT。IDT有256个表项, 每个表项对应一个中断, 是一个中断门描述符, 登记中断处理程序的入口地址。
- CPU内部, 有一个IDTR寄存器, Base Address和 Limit分别是IDT的首地址和长度。

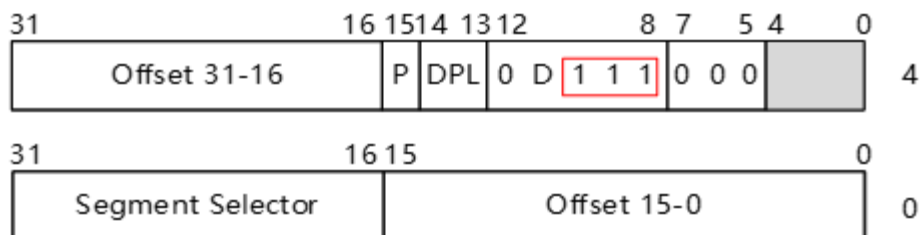
中断门描述符

中断门



Segment Selector, 段选择子。中断入口程序在核心态代码段, 0x8。
Offset, 中断入口程序的虚地址。

陷入门



外部中断使用中断门 (110)。响应中断时, 硬件关中断。
∴ 外部中断处理程序, 关中断状态运行。

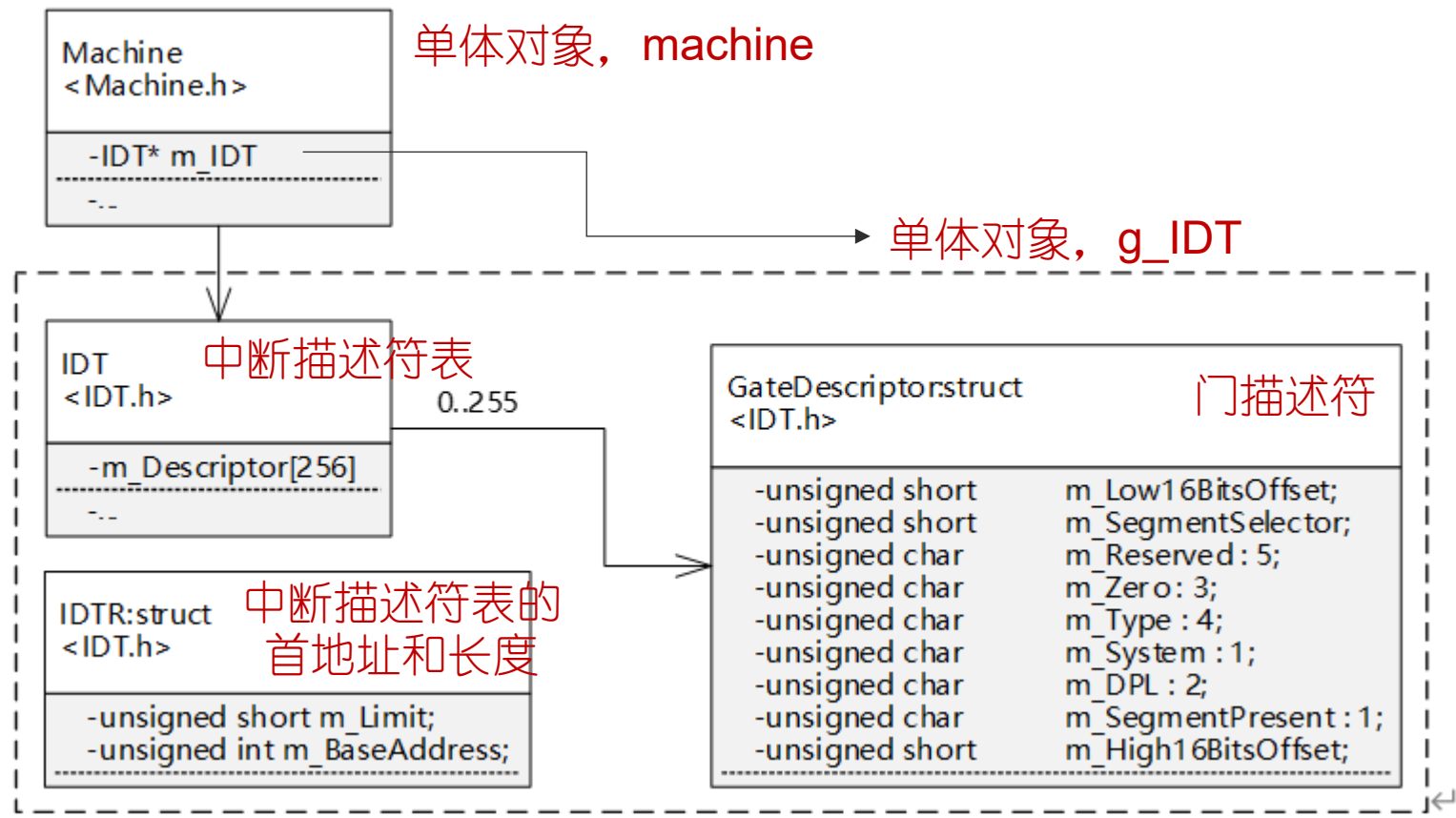
异常和系统调用使用陷入门 (111)。响应中断时, 硬件不关中断。
∴ 异常处理程序 和 系统调用处理程序, 开中断运行。



中断号的分配

- 0~31, 异常。0~19有定义, 20以上没有定义
- 32~255, 其它中断号
 - 留1个给内核, 用来实现系统调用。我们的, 是0x80
 - 应用程序执行 `int 0x80` 指令, 激活内核执行系统调用入口程序
 - 大的中断号, 留几个, 用来实现处理器之间中断
 - 其余, 按需分配给外设和时钟

Unix V6++，中断系统的核心数据结构



内核初始化时，(1) 写 g_IDT 中的中断描述符表 (2) 写 IDTR 结构，并且将其加载入 CPU，寄存器 IDTR，让 CPU 引用内存中的中断描述符表。

kernel/main.cpp

```
62 //init idt
63 machine.InitIDT();
64 machine.LoadIDT();
```

初始化完毕后，系统开始响应中断请求。系统正常运行时，IDT 常驻内存，值不变。CPU 始终用它响应中断请求。



中断描述符表

已定义异常，Unix
默认异常处理方式

包含堆栈自动扩展分支

CPU未定义异常，
Panic，系统停摆，等待
系统管理员关机、重启

中断号	语义	门类型	段选择子	offset (中断入口程序首地址)
0x0	除0错	陷入门	0x8	Exception::DivideErrorEntrance()
0x1	调试异常	陷入门	0x8	Exception::DebugEntrance()
0x2	NMI非屏蔽中断	陷入门	0x8	Exception::NMIEntrance()
0x3	调试断点	陷入门	0x8	Exception::BreakpointEntrance()
0x4	溢出	陷入门	0x8	Exception::OverflowEntrance()
0x5	BOUND指令异常	陷入门	0x8	Exception::BoundEntrance()
0x6	无效操作码	陷入门	0x8	Exception::InvalidOpcodeEntrance()
0x7	设备不可用	陷入门	0x8	Exception::DeviceNotAvailableEntrance()
0x8	双重错误	陷入门	0x8	Exception::DoubleFaultEntrance()
0x9	协处理器段越界	陷入门	0x8	Exception::CoprocesorSegmentOverrunEntrance()
0xA	无效TSS	陷入门	0x8	Exception::InvalidTSSEntrance()
0xB	段不存在	陷入门	0x8	Exception::SegmentNotPresentEntrance()
0xC	堆栈段错误	陷入门	0x8	Exception::StackSegmentErrorEntrance()
0xD	一般保护性异常	陷入门	0x8	Exception::GeneralProtectionEntrance()
0xE	缺页异常	陷入门	0x8	Exception::PageFaultEntrance()
.....				
0x14~0x1F	保留	陷入门	0x8	IDT::DefaultExceptionHandler

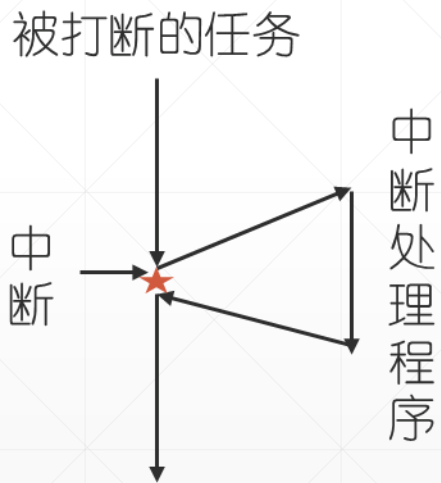


中断描述符表

中断号		语义	门类型	段选择子	offset（中断入口程序首地址）
外部中断	0x20	时钟中断	中断门	0x8	Time::TimeInterruptEntrance()
	0x21	键盘中断	中断门	0x8	KeyboardInterrupt::KeyboardInterruptEntrance()
	...	...		0x8	IDT::DefaultInterruptHandler()
	0x2E	硬盘中断	中断门	0x8	DiskInterrupt::DiskInterruptEntrance()
	...	...		0x8	IDT::DefaultInterruptHandler()
系统调用	0x80		陷入门	0x8	SystemCall::SystemCallEntrance()

.....
CPU未定义中断，死循环。等待系统管理员关机、重启

四、中断处理流程（T0时刻，PA是现运行进程）



■ CPU硬件

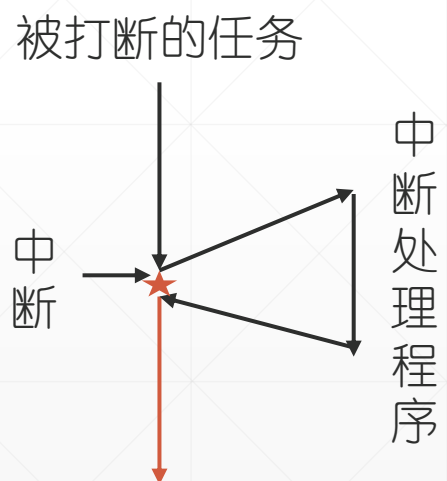
- 保护中断发生时的CPU执行现场，将其压入现运行进程核心栈（第1现场保护区）
- 用中断号查中断向量表，得到门描述符
- 将门描述符中的： 段选择子(0x8)→CS offset→EIP

强制CPU陷入核心态，跳转执行中断入口程序

■ 软件

现运行进程（PA）执行内核中断处理逻辑

- 将剩余寄存器压入核心栈（形成第2现场保护区），完成保护所有段寄存器、通用寄存器的任务
- 调用中断处理程序，传入第1，2现场保护区的首地址
 - 中断号如果是0x80，这里就是内核在执行系统调用
 - 注意：中断，经常会改变调度系统的状态。所以，中断处理结束后，现运行进程极有可能不再是优先级最高的进程



▪ 软件

▪ 中断返回。

- 如果现运行进程不再是优先权最高的进程，执行 Swtch()放弃CPU
- 从核心栈中恢复被中断现场，继续执行被打断的任务
 - 执行一系列pop指令，恢复第2现场
 - IRET (恢复第1现场)

被中断的进程继续执行被打断的任务

五、中断对调度系统的影响



六、源代码 和 情景分析

- T时刻，现运行进程PA，用户态运行，正在执行应用程序。CPU响应磁盘中断请求。

CPU现场

段寄存器

最后2个bit, 是11

- CS = 0x1b
- DS = 0x23
- ES = 0x23
- SS = 0x23
- GS = 0x23
- FS = 0x23

磁盘中断处理 程序入口地址

段描述子: 0x8

偏移量: DiskInterrupt::DiskInterruptEntrance()

EFLAGS

CPU开中断

IF标识

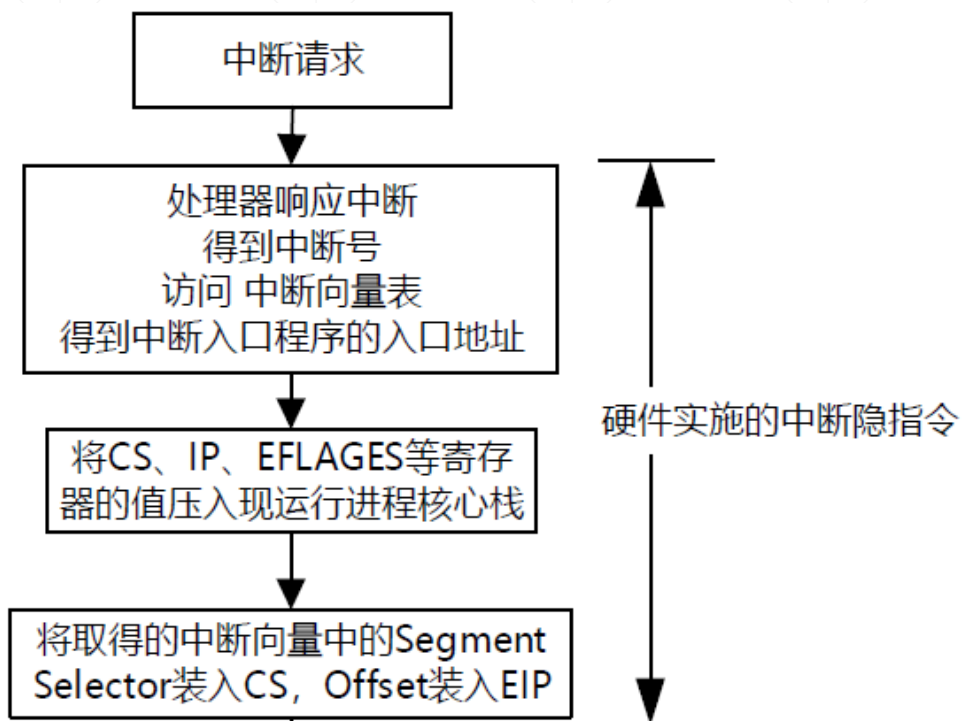


通用寄存器（应用程序的执行状态）：

- EIP 0x401006
- ESP、EBP 用户栈，栈顶帧
-

1、CPU硬件响应中断，启用核心栈，保存现场

当前指令执行完毕后，响应磁盘中断请求



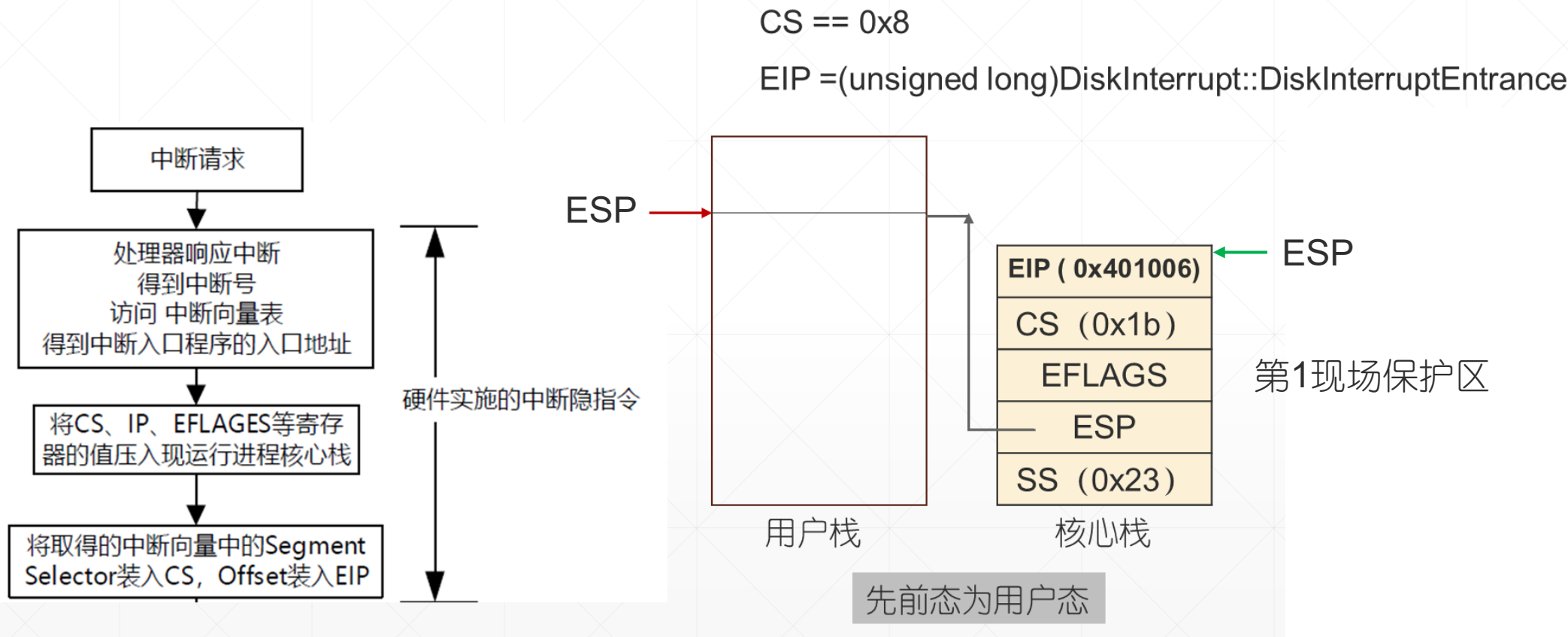
每颗CPU有自己的TSS段，这是核心态数据结构。其中有两个重要字段：

- SS0，核心栈所在段的段选择子，值是0x10。
- esp0，现运行进程核心栈底的线性地址，值是0xC0400000。

处理器响应中断：当前

- 用户态执行（CPL==11）：暂存 SS 和 ESP（用户态的）。从TSS段中取出来 SS0 和 esp0 赋值 SS 和 ESP 寄存器。这就成功启用了现运行进程的核心栈。之后，push暂存的 SS和ESP。push EFLAGS，CS 和 EIP 寄存器。这就压栈保存了进程最重要的用户态执行现场。
- 核心态执行（CPL==00）：直接 push EFLAGS，CS 和 EIP。因为响应中断不需要切换堆栈，所以不用push SS 和 ESP。

切换至核心态、CPU跳转至磁盘中断处理程序入口



2. 软件响应中断 中断入口程序 和 中断处理程序

```
void DiskInterrupt::DiskInterruptEntrance()  
{
```

```
    SaveContext();           // 继续压栈保护通用寄存器（第2现场保护区）  
                             & 为磁盘中断处理程序传参
```

```
    SwitchToKernel();        // 继续设置段寄存器
```

```
    CallHandler(ATADriver, ATAHandler);  
    // 调用磁盘中断处理程序
```

```
    L: ..... 例行调度 .....
```

```
    RestoreContext();        // 恢复现场（第2现场保护区）  
    Leave();                 // 让ESP回到第1现场保护区  
    InterruptReturn();        // 中断返回（恢复现场，第1现场保护区）
```

```
}
```

```
void ATADriver::ATAHandler(  
    struct pt_regs *reg, struct pt_context *context)  
{  
    // 处理IO完成事件  
    // 唤醒等待IO完成的进程  
}
```

2.1 SaveContext()

```
#define SaveContext() \
    __asm__ __volatile__ ("    cld; \
    pushl %%eax; \
    pushl %%ebp; \
    pushl %%edi; \
    pushl %%esi; \
    pushl %%edx; \
    pushl %%ecx; \
    pushl %%ebx; \
    pushl %%es; \
    pushl %%ds; \
    pushl %%fs; \
    pushl %%gs; \
    lea 0x4(%%ebp), %%edx; \
    pushl %%edx; \
    lea 0x4(%%esp), %%edx; \
    pushl %%edx");
```

// 压栈 EBP+4 的值

// 压栈 ESP+4 的值

为中断处理程序准备的实参

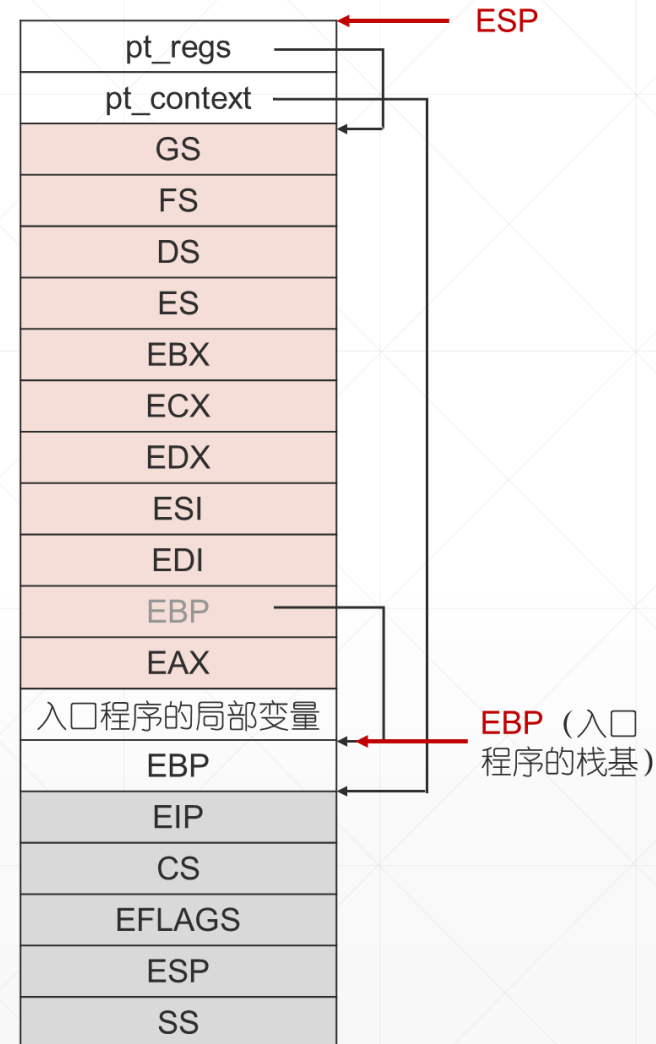
第2现场保护区

C函数DiskInterruptEntrance()入口，gcc会插入以下3条指令，对核心栈产生作用：

```
push %ebp
mov %esp, %ebp
sub $***, %esp
```

回顾，showStack程序的子函数sum()

第1现场保护区



2.2

SaveContext();

```
#define SwitchToKernel() \
    __asm__ __volatile__ (" mov $0x10, %%dx; \
                           mov %%dx, %%ds; \
                           mov %%dx, %%es";);
```

CallHandler(ATADriver, ATAHandler);

```
#define CallHandler(Class, Handler) \
    __asm__ __volatile__ (" call *%%eax" :: "a" (Class::Handler) );
```

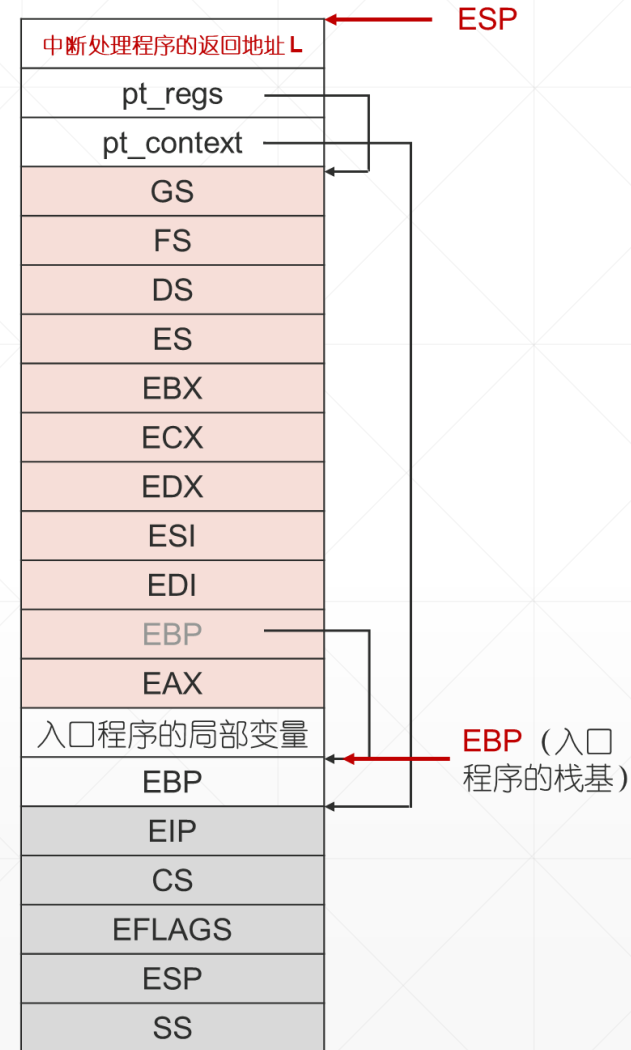
▪ 核心态

- CS = 0x8
- DS = 0x10
- ES = 0x10
- SS = 0x10

为中断处理程序准备的实参

第2现场保护区

第1现场保护区





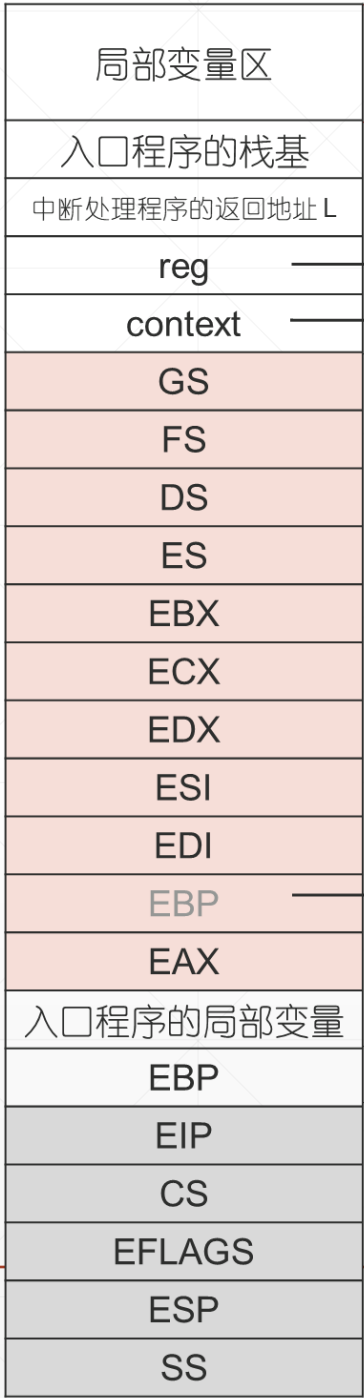
2.3 执行中断处理程序

```
void ATADriver::ATAHandler(  
    struct pt_regs *reg, struct pt_context *context)  
{  
    处理IO完成事件  
    唤醒等待IO完成的进程, RunRun++  
  
    // context -> xcs 是 中断前CS寄存器的值  
    // reg -> eax 是 中断前eax寄存器的值  
}
```

中断处理程
序的栈帧

第2现场
保护区

第1现场
保护区



ESP

EBP

```
7 struct pt_regs{  
8     unsigned int pad1;  
9     unsigned int pad2;  
10    unsigned int xds;  
11    unsigned int xes;  
12    unsigned int ebx;  
13    unsigned int ecx;  
14    unsigned int edx;  
15    unsigned int esi;  
16    unsigned int edi;  
17    unsigned int ebp;  
18    unsigned int eax;  
19 };
```

入口程序的栈基

```
27 struct pt_context{  
28     unsigned int eip;  
29     unsigned int xcs;  
30     unsigned int eflags;  
31     unsigned int esp;  
32     unsigned int xss;  
33 };
```


2.4 例行调度

```

struct pt_context *context;
__asm__ __volatile__ ("    movl %%ebp, %0; addl $0x4, %0 " : "+m" (context) );
                                context 指针的值是 ebp+4

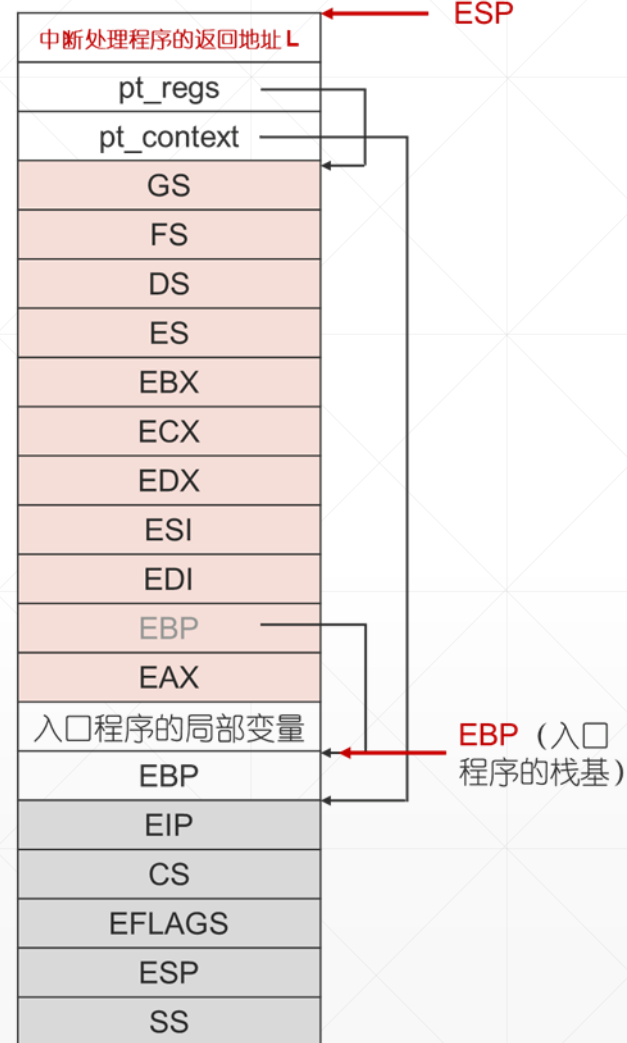
if( context->xcs & USER_MODE ) /*先前为用户态*/
{
    L1: while(true)
    {
        X86Assembly::CLI(); /* 关中断*/

        if(Kernel::Instance().GetProcessManager().RunRun > 0)
        {
            X86Assembly::STI(); /* 开中断*/
            Kernel::Instance().GetProcessManager().Swch(); /* 现运行进程放弃CPU */
        }
        else
            break; /* 如果runrun == 0, 则退栈回到用户态继续用户程序的执行 */
    }
}
    
```

为中断处理程序准备的实参

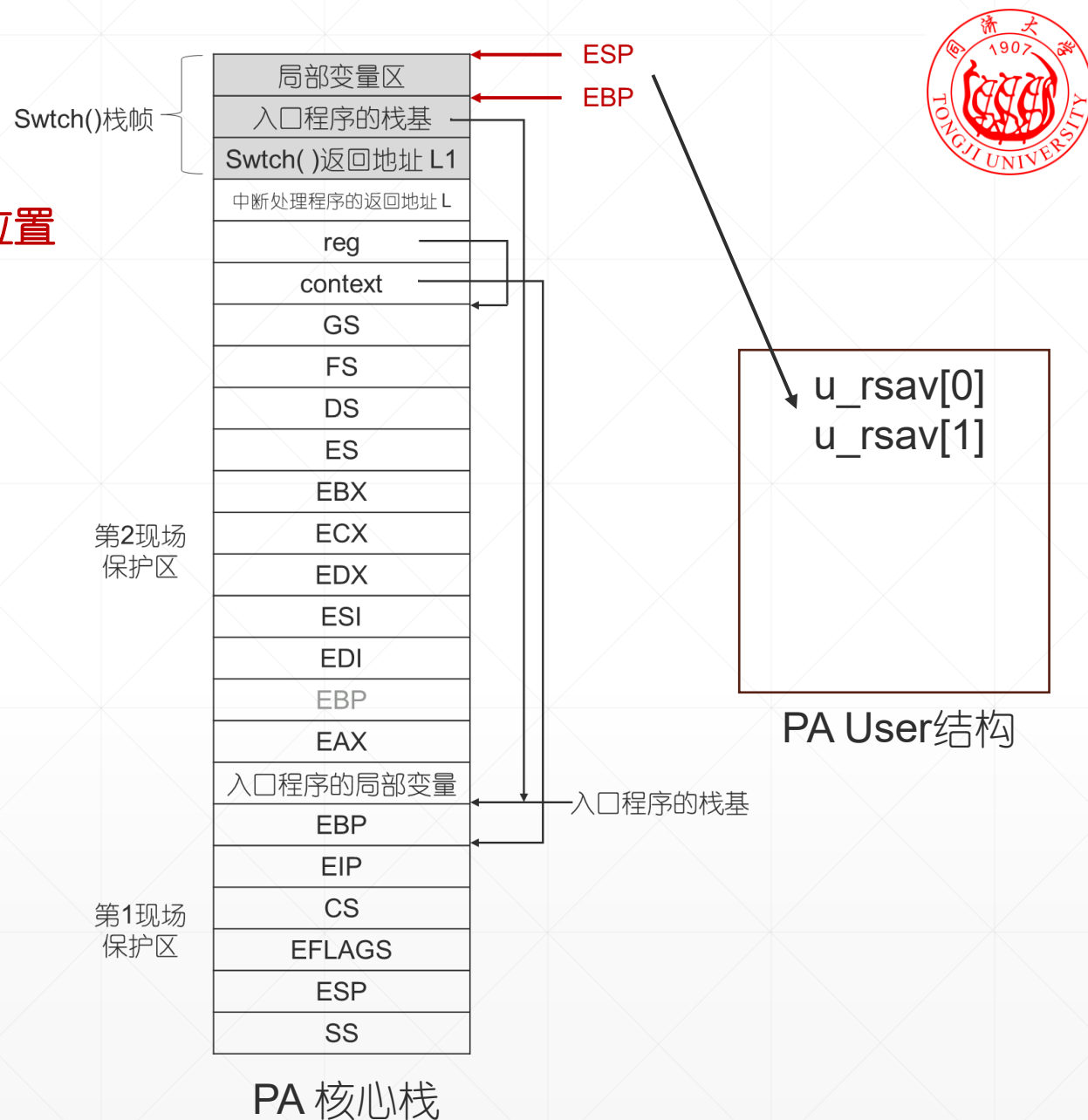
第2现场保护区

第1现场保护区



int ProcessManager::Switch()

```
{  
    User& u = Kernel::Instance().GetUser();  
    SaveU(u.u_rsav); // 1、保护PA, Switch栈帧的位置  
  
    Process* procZero = &process[0];  
    X86Assembly::CLI();  
    SwtchUStruct(procZero);  
    RetU();  
    X86Assembly::STI();  
    Process* selected = Select();  
  
    X86Assembly::CLI();  
    SwtchUStruct(selected);  
    RetU();  
    X86Assembly::STI();  
    User& newu = Kernel::Instance().GetUser();  
    newu.u_MemoryDescriptor.MapToPageTable();  
  
    return 1;  
}
```



SaveU(u.u_rsav)

```
int ProcessManager::Swch()
```

```
{
```

```
    User& u = Kernel::Instance().GetUser();    // PA 的user结构
```

```
    SaveU(u.u_rsav);
```

```
    .....
```

```
}
```

宏展开后，是两条指令

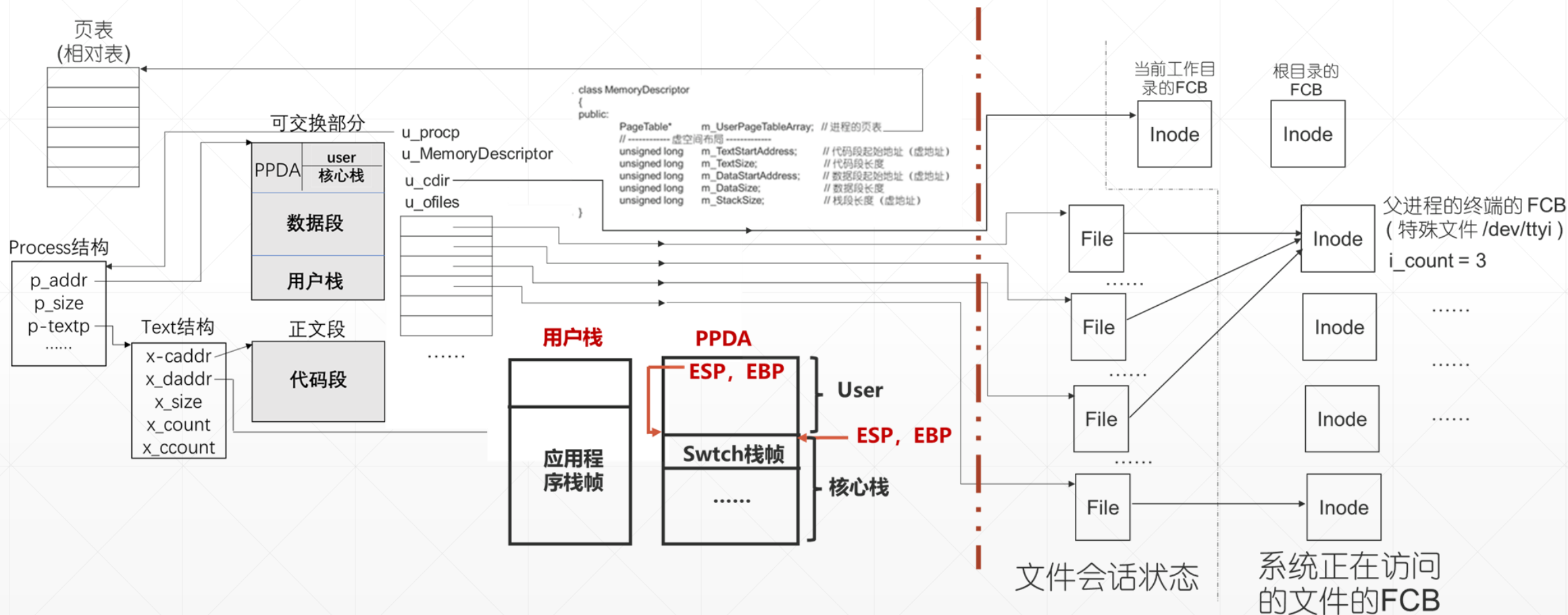
```
movl %esp, u.u_rsav[0]
```

```
movl %ebp, u.u_rsav[1]
```

```
#define SaveU(u_sav) \
    __asm__ __volatile__( \
        "movl %%esp, %0\n\t \
        movl %%ebp, %1\n\t" \
        : "+m"(u_sav[0]), "+m"(u_sav[1]) \
        );
```

```
class User
{
public:
    unsigned long u_rsav[2]; /* 用于保存esp与ebp指针 */
    .....
```

Swch()为下台进程制作的可执行图像





int ProcessManager::Swch()

```
{
    User& u = Kernel::Instance().GetUser();
    SaveU(u.u_rsav);

    Process* procZero = &process[0];
    X86Assembly::CLI();
    SwchUStruct(procZero); // 映射0#进程的PPDA。user、核心栈可见
    RetU();                // ESP、EBP指向0#进程的栈顶帧 Swch
    X86Assembly::STI();
    Process* selected = Select(); // 2、0#进程执行Select( )函数，挑选新运行进程 selected

    X86Assembly::CLI();
    SwchUStruct(selected);
    RetU();
    X86Assembly::STI();
    User& newu = Kernel::Instance().GetUser();
    newu.u_MemoryDescriptor.MapToPageTable();

    return 1;
}
```

Select()函数选新运行进程

算法将Process数组维护成循环队列。

lastSelect是当前指针，现运行进程的下标。

- 系统初始化时，置0。
- 进程切换时，从下个元素开始，寻找图像在内存中的、优先级最高的就绪进程。

```

static int lastSelect = 0;
while (true)
{
    this->RunRun = 0;

    int priority = 256;
    int best = -1;
    for ( int count = 0; count < NPROC ; count++ )
    {
        int i = (lastSelect + 1 + count) % NPROC;
        if ( Process::SRUN == process[i].p_stat && (process[i].p_flag & Process::SLOAD) != 0 )
            if ( process[i].p_pri < priority )
            {
                best = i;
                priority = process[i].p_pri;
            }
    }
    if ( -1 == best )
    {
        __asm__ __volatile__ ("hlt");
        continue;
    }

    this->CurPri = priority;
    lastSelect = best;
    return &process[best];
}

```

1、遍历Process数组，寻找优先级最高（p_pri值最小）的进程 best

Select函数运行在0#进程核心栈。

2、返回选中进程Process结构的首地址

int ProcessManager::Swtch()

```
{
```

```
User& u = Kernel::Instance().GetUser();  
SaveU(u.u_rsav);
```

```
Process* procZero = &process[0];  
X86Assembly::CLI();  
SwtchUStruct(procZero); // 映射0#进程的PPDA。user、核心栈可见  
RetU(); // ESP、EBP指向0#进程的Swtch栈帧  
X86Assembly::STI();  
Process* selected = Select();
```

```
X86Assembly::CLI(); // 3、恢复选中进程的执行状态
```

```
SwtchUStruct(selected);
```

```
RetU(); // 3.1 从user结构取出保存的值。恢复Swtch( )栈帧  
// 让ESP、EBP重新指向新选中进程的Swtch栈帧
```

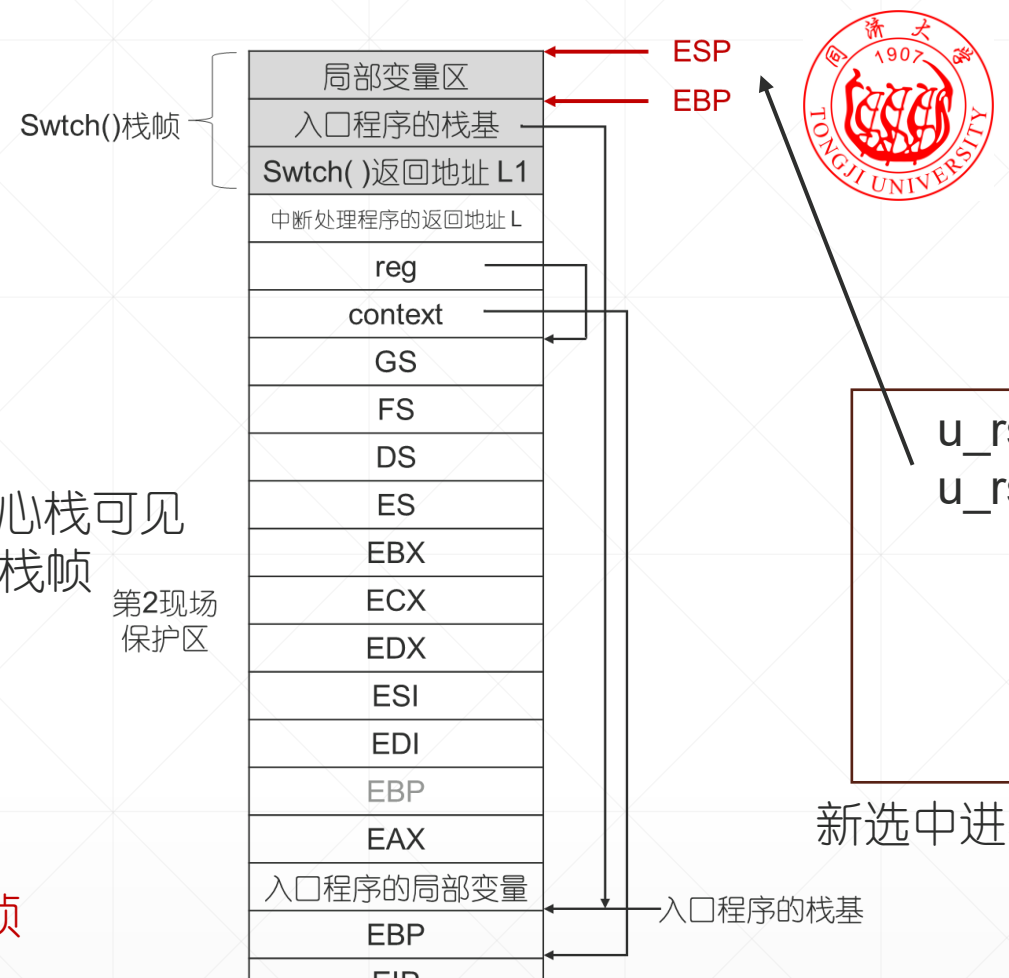
```
X86Assembly::STI();
```

```
User& newu = Kernel::Instance().GetUser(); // 引用该进程的user结构
```

```
newu.u_MemoryDescriptor.MapToPageTable(); // 3.2 取出相对表，为用户空间建立地址映射关系
```

```
return 1;
```

```
}
```



新选中进程的核心栈



回到中断入口程序

```
struct pt_context *context;
__asm__ __volatile__ ("    movl %%ebp, %0; addl $0x4, %0 " : "+m" (context) );
                                context 指针的值是 ebp+4
if( context->xcs & USER_MODE ) /*先前为用户态*/
{
    0x3
L1: while(true)
    {
        X86Assembly::CLI(); /* 关中断*/

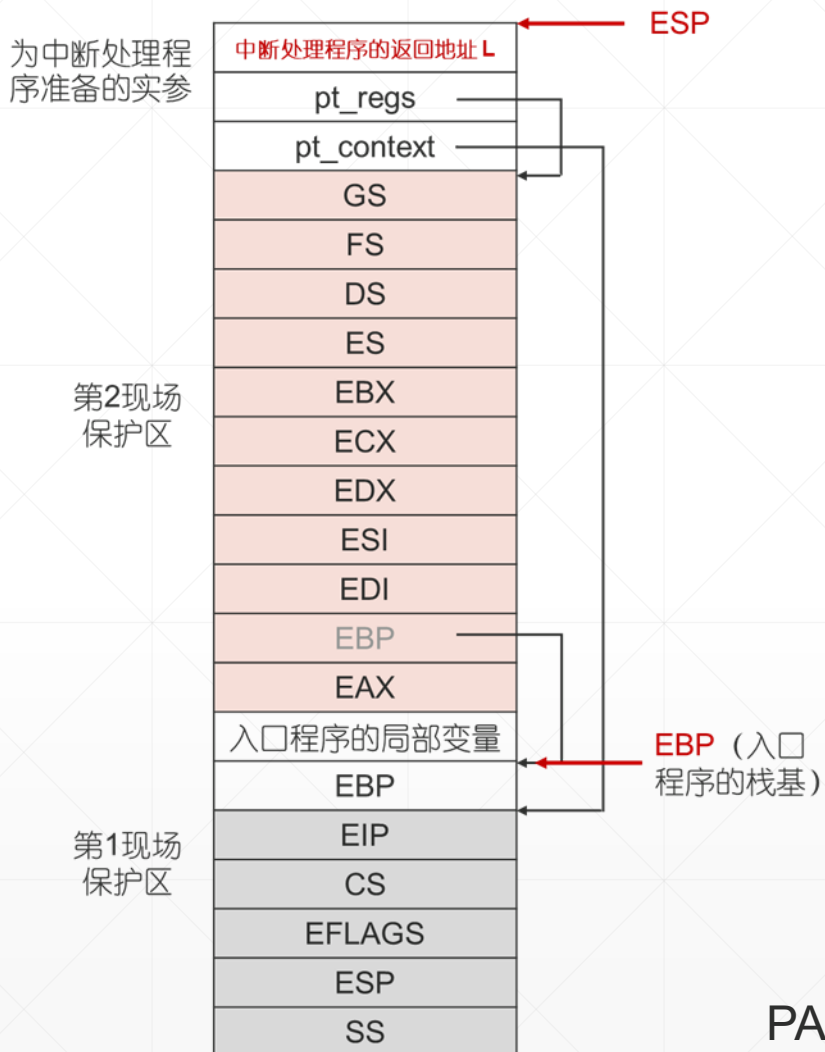
        if(Kernel::Instance().GetProcessManager().RunRun > 0)
        {
            X86Assembly::STI(); /* 开中断*/
            Kernel::Instance().GetProcessManager().Swch(); /* 现运行进程放弃CPU */
        }
        else
            break; /* 如果runrun == 0, 则退栈回到用户态继续用户程序的执行 */
    }
}
```


2.5 恢复被中断现场 继续执行被打断的任务

`RestoreContext();`
恢复现场（第2现场保护区）

`Leave();`
(1) 恢复中断发生时的EBP寄存器
(2) ESP回到第1现场保护区

`InterruptReturn();`
恢复现场（第1现场保护区）



PA继续执行被打断的应用程序

总结：可剥夺调度策略， Unix V6的实现方式

